Sistemas Paralelos - 2026

Trabajo Práctico Nº 1

Punteros y gestión de memoria en C

Facultad de Informática

Objetivos de la práctica: Comprender la asignación de memoria dinámica y el direccionamiento mediante punteros para implementar programas que requieran utilizar grandes estructuras de datos.

1. Punteros

Un puntero es una variable cuyo contenido es una dirección de memoria. En la declaración de un puntero se indica el tipo del objeto de datos que se encontrará a partir de la dirección dada. Por ejemplo, la sentencia: `int *p;` declara el puntero *p* que apunta a un objeto de datos de tipo *int*.

Para obtener la dirección de memoria de un objeto de datos se agrega el operador & delante del identificador del objeto. Ejemplo:

`int a = 33;` `p = &a;`	se almacena en *p* la dirección de memoria de la variable *a*, siendo *p* la variable de tipo puntero declarada anteriormente.

Para acceder al objeto de datos apuntado por un puntero, se agrega el operador * antes del mismo. Por ejemplo:

`int x;` `x = a + *p;`	asigna a la variable *x* el resultado de *a* + ∗*p*, es decir, de *a*+*a*, siendo *a* y *p* las variables anteriormente declaradas.

Es importante aclarar que el acceso a datos a través de punteros debe hacerse en espacios de memoria pertenecientes a objetos de datos ya declarados por el programa. Ejemplo:

`int *y;` `*y = 10;`	en este caso, se declara *y* como un puntero a un entero, y se asigna el valor 10 al objeto de datos apuntado por *y*. La compilación no da errores, pero se encontrarán problemas en la ejecución ya que *y*, en vez de contener una dirección válida de algún objeto de datos ya declarado, contiene un valor indefinido.

Ejercicios:

1.1) Dado el siguiente código en C, resolver:

a) Crear, utilizando el editor de texto nano (o vim), el código fuente con el nombre `archivo.c`

b) Compile el programa:
`$ gcc archivo.c -o prog`

c) Ejecútelo:
`$ ./prog`

d) Razone el resultado.

```
#include <stdio.h>
int main() {
   int x = 10;
   int *p;
   p = &x;
```

```
   printf("Dirección de x: %p \n", &x);
   printf("Dirección almacenada en p: %p \n", p);
   printf("Valor de *p: %d \n", *p);

   return 0;
}
```

1.2) Dado el siguiente código en C, resolver:

a) Analice el código del programa para determinar el valor final de *a* y *b*.
b) Compile el programa y ejecútelo para comprobar los resultados.

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main() {
   int a, b;
   int *x, *y;

   a = 10;
   b = 20;
   x = &a;
   y = &b;
   x = y;
   *x = 5;

   return 0;
}
```

2. Punteros y arreglos

El nombre de un arreglo se evalúa a su dirección inicial (*`arreglo`* es equivalente a *`&arreglo[0]`*), es decir, es un puntero al inicio del arreglo. De esta manera, es posible operar sobre arreglos utilizando la notación de arreglos o punteros indistintamente. Por ejemplo, dado el arreglo `int a[10]`, el primer elemento puede ser accedido indistintamente como `a[0]` o `*a`, el segundo elemento como `a[1]` o `*(a+1)`,..., y el décimo como `a[9]` o `*(a+9)`. Lo mismo ocurre a la inversa, es decir, es posible operar con un puntero utilizando la notación de punteros o arreglos indistintamente.

Ejercicios

2.1) Declare un arreglo *a* de 5 elementos de tipo *float*. Luego asigne cualquier valor al contenido del quinto elemento, accediéndolo a través de la notación de punteros. Finalmente muestre por pantalla el valor del elemento modificado utilizando la notación de arreglos.

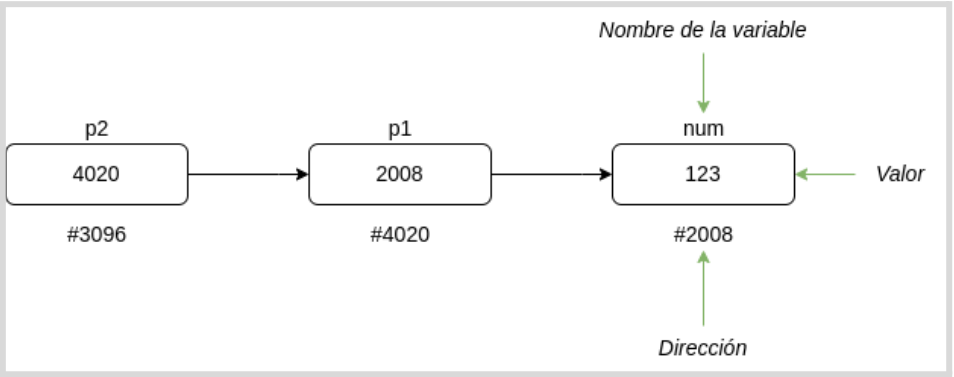
2.2) Este ejercicio se basa en descubrir cómo se almacena una matriz en memoria. Resuelva:

a) Escriba un programa en C que declare una matriz de 3x4 elementos enteros, y muestre la dirección para cada elemento de la matriz. La salida debe ser: `"Elemento[i][j] - Dirección"`.

b) A partir de los valores de direcciones obtenidos para cada elemento, deduzca la disposición de los elementos de la matriz en memoria: almacenamiento por filas o almacenamiento por columnas.

3. Punteros a punteros

Un puntero a puntero es una forma de indirección múltiple. Normalmente, un puntero contiene la dirección de una variable objetivo que es de un cierto tipo que no es justamente un puntero. Cuando definimos un puntero a puntero, el primer puntero contiene la dirección del segundo puntero, el cual contiene la dirección de la variable objetivo. Esto se puede observar en la siguiente figura:



Cuando se desea declarar un puntero a puntero, se ubica un asterisco adicional en el frente de su nombre. Por ejemplo: `int **p2;`

Ejercicio

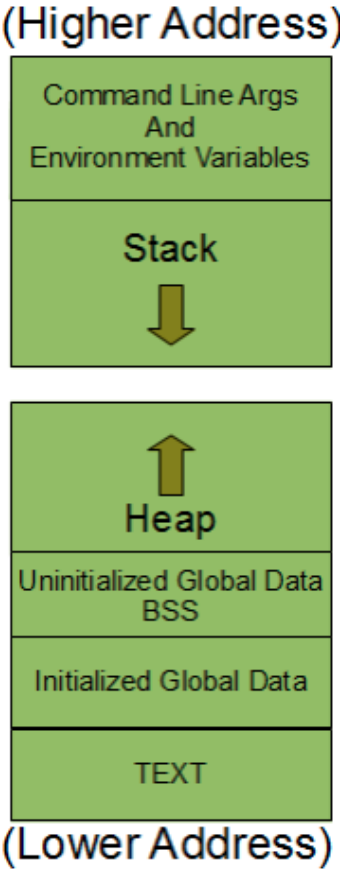
3.1) Dado el siguiente código en C. ¿Cuál cree que será el valor final de *num*? Compile el programa y ejecútelo para comprobar el resultado.

```
#include <stdio.h>

int main() {
   int num;
   int *p1;
   int **p2;

   num = 123;
   p1 = &num;
   p2 = &p1;

   *p1 = num - 23;
   **p2 = *p1 * 2;

   printf("Valor de num = %d\n", num);

   return 0;
}
```

4. Organización de la memoria

Los programas en ejecución (procesos), son organizados o desplegados en segmentos de memoria de diferentes tamaños y con distintas propiedades. Esta organización en segmentos de memoria de los programas dependen fuertemente del compilador, sistema operativo y hardware subyacente para los cuales fueron construidos.

Un proceso de un programa en C típicamente consiste de los siguientes segmentos:



Stack
Este segmento contiene la pila del programa, una estructura LIFO (último que entra primero que sale). Cuando se llama a una función, se almacena en la pila la dirección de retorno y cierta información sobre el entorno de la función que hace la llamada (por ejemplo los valores de registros que quieran restaurarse cuando se retorne de la función). Luego, se asignará un espacio en la pila para almacenar las variables de la función que se ha llamado. Una vez que la función termina de ejecutarse, se quitarán de la pila todos los datos que se asignaron cuando se realizó su llamada. Este esquema permite el uso de funciones recursivas dado que cada vez que una función recursiva se llama a sí misma, se utiliza un nuevo espacio de la pila para almacenar las nuevas variables, evitando que interfieran con las variables de otra instancia de la función. El espacio de memoria reservado para el Stack es fijo y determinado al momento de creación del proceso (normalmente de 8MB, y se puede conocer ejecutando el comando ulimit -s).

Heap
Este segmento es el que se utiliza para la asignación de memoria dinámica, es decir, memoria que el proceso solicita al sistema operativo en tiempo de ejecución. Tan pronto el proceso deje de usar el nuevo espacio de memoria solicitado, el mismo proceso deberá explícitamente liberarlo para que otros procesos o él mismo pueda reutilizar. En C no hay un recolector de basura como sí existe en otros lenguajes. Esto hace que C sea más eficiente, sin embargo el programador deberá ser mucho más cuidadoso para gestionar la memoria dinámica.

Uninitialized Data o BSS
Este segmento contiene todas las variables globales y variables estáticas que no fueron inicializadas o fueron inicializadas en cero (todos sus bits están en cero).

Initialized Data
En este segmento se almacenan todas las variables globales o variables estáticas que fueron inicializadas con valores distintos de cero.

Text
El segmento de texto es el área de memoria que contiene las instrucciones de máquina que ejecuta la CPU.

5. Asignación dinámica de memoria

Al declarar un arreglo que utilizan memoria estática (del segmento Stack, BSS, o Initialized Data), cuando el tamaño del arreglo excede el tamaño del segmento estático utilizado, la ejecución dará un error de violación de segmento. Por esta razón, para almacenar arreglos grandes es necesario utilizar la asignación dinámica de memoria (segmento Heap).

La gestión dinámica de memoria se lleva a cabo mediante las funciones de *malloc* y *free*, que permiten solicitar al S.O. un bloque de memoria contiguo y liberarlo, respectivamente. La función *malloc(n)* reserva un bloque de memoria de *n* bytes y retorna un puntero al inicio del bloque. La función *free(p)* libera el espacio de memoria apuntado por el puntero *p*, que debe haber sido retornado por una llamada previa a *malloc*. Nunca liberar dos veces un mismo bloque de memoria, ya que el comportamiento es indefinido.

El programa debería verificar que *malloc* no devuelva *NULL* por ser imposible satisfacer el requerimiento de memoria. El símbolo *NULL* corresponde a un puntero nulo, es decir, no es una dirección utilizable.

Ejercicios

5.1) Explique qué hace el siguiente fragmento de código:

```
int *p;
p = (int *) malloc(sizeof(int));
*p = 50;
```

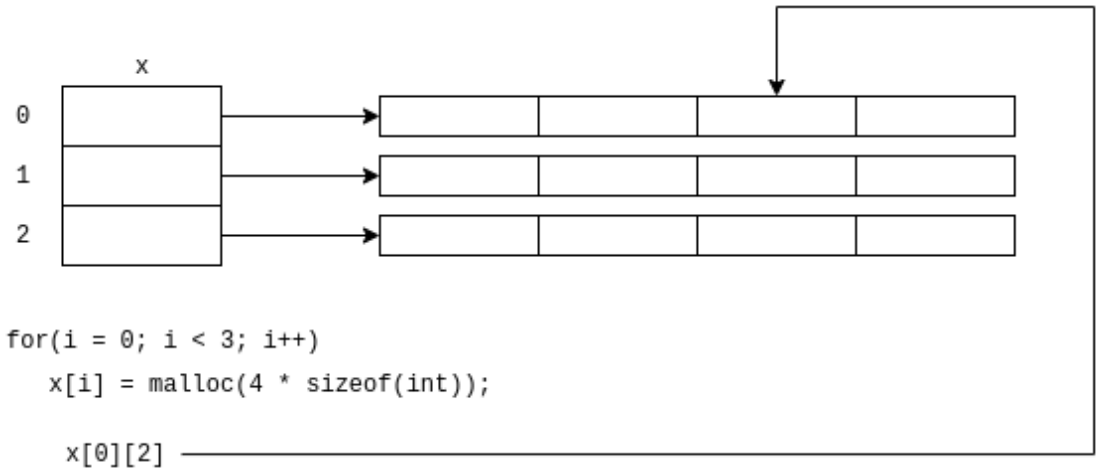
5.2) Reemplazar la sentencia **"int p[5]"** por una declaración con memoria dinámica.

```
int p[5];
p[0] = 1;
p[1] = 2;
p[2] = 3;
p[3] = 4;
p[4] = 5;
```

5.3) Almacenamiento de matrices con memoria dinámica

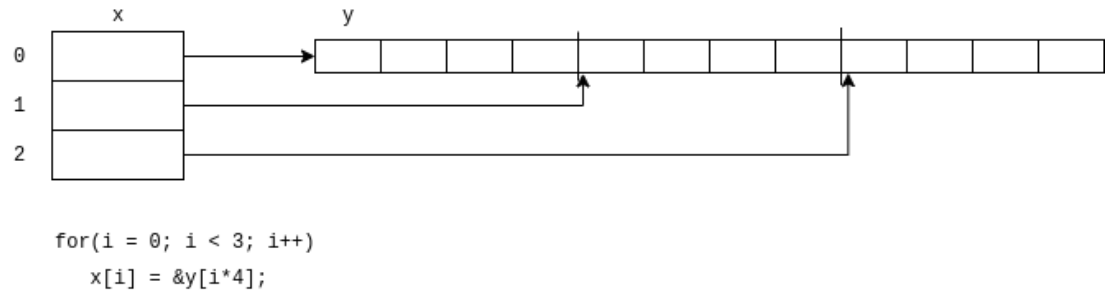
Es posible almacenar las matrices utilizando las siguientes dos estrategias.

a) **Almacenamiento con filas separadas en memoria**



En este caso, utilizamos un arreglo de punteros *x*, donde cada uno de sus elementos mantiene la dirección del primer byte de la fila a la que apunta. Como se puede observar en la figura, hay un bucle en el que para cada `x[i]` se utiliza la función *malloc* solicitando espacio para 4 elementos de tipo entero. Recordemos que esta función reserva el bloque de memoria solicitado y retorna un puntero al inicio del bloque.

b) Almacenamiento con filas unidas en memoria



Para esta forma de almacenamiento de matrices, se reserva un único bloque de memoria contiguo que permita almacenar todas las filas de la matriz, una después de otra. A su vez, se declara un arreglo de punteros *x* en el cual se almacena las direcciones de memoria del primer byte de cada fila (dirección del elemento `y[i*columnas]`).

Resolver: implementar un programa que, utilizando el almacenamiento con filas unidas, almacene e imprima la siguiente matriz de 3x4

0	1	2	3
1	2	3	4
2	3	4	5

c) Espacio máximo de memoria RAM disponible

Utilizando el comando ***free***, determinar:

i) El tamaño máximo de memoria que podría solicitar un proceso evitando el uso del área de intercambio.

ii) Suponga una estructura de datos que almacena una gran matriz de elementos de tipo *double*. Determinar el número máximo aproximado de elementos que podría tener la matriz para el espacio de memoria calculado en el inciso anterior.